

Comparative Analysis of Reinforcement Learning Algorithms on FrozenLake Environments

Aniket Mandal

June 2025

1 Introduction

This report presents an in-depth conceptual comparison of five foundational Reinforcement Learning (RL) algorithms: Value Iteration, Policy Iteration, Q-Learning, SARSA, and Monte Carlo. These methods were applied to various FrozenLake environments including the standard 8x8 environment, a custom-generated 100x100 map, and an expanded version featuring a teleportation action. The primary goal is to analyze how different algorithms adapt to increasing complexity and changing dynamics while maximizing cumulative rewards.

2 Environment Setup

The experimentation was conducted using the Gymnasium library in Python. Three types of FrozenLake environments were defined:

- **Original FrozenLake (8x8)**: A standard, deterministic grid world.
- **CustomFrozenLake (100x100)**: A large, randomly generated deterministic map.
- **ExpandedFrozenLake (100x100)**: An extended version of CustomFrozenLake including a fifth action for teleportation to a random state.

Each environment was registered with a high maximum episode step count (10,000) to accommodate the scale.

3 Algorithm Descriptions

3.1 Value Iteration

This algorithm iteratively updates the value of each state based on the maximum expected reward over all possible actions. It relies on full knowledge of the transition dynamics and rewards. Its deterministic approach allows it to find the optimal policy through successive approximations until convergence.

3.2 Policy Iteration

Policy Iteration involves two alternating phases: policy evaluation and policy improvement. The evaluation phase estimates the value of a current policy, while the improvement phase updates the policy based on the estimated values. This process repeats until the policy stabilizes and becomes optimal.

3.3 Q-Learning

Q-Learning is a model-free method where the agent learns action-value estimates (Q-values) directly from experience. It selects actions using an ϵ – greedy strategy and updates its Q – table based on observed transitions. It eventually converges to the optimal policy as the exploration diminishes.

3.4 SARSA

SARSA is an on-policy counterpart to Q-Learning. It updates the Q-value based on the action that was actually taken rather than the estimated best action. This results in more cautious learning, potentially avoiding risky paths in stochastic or complex environments.

3.5 Monte Carlo

Monte Carlo methods compute value estimates by averaging returns from complete episodes. It is suitable for episodic tasks and requires no knowledge of the transition model. Its learning is delayed until the end of each episode, making it less sample-efficient but conceptually straightforward.

4 Evaluation Methodology

Each learned policy was evaluated by running it over 100 episodes in the respective environments. The metric used was the average reward across episodes, reflecting how well the learned policy performs in the long run.

5 Performance Results

Environment	Value Iteration	Policy Iteration	Q-Learning	SARSA	Monte Carlo
Original 8x8	1.000	1.000	0.000	0.000	0.000
Custom 100x100	1.000	1.000	0.000	0.000	0.000
Expanded 100x100	0.020	0.020	0.000	0.000	0.000

6 Analysis and Discussion

6.1 Small Environment (8x8)

In the small environment, both Value and Policy Iteration achieve perfect performance. These results reflect their inherent advantage in environments with manageable state spaces where transition dynamics are fully known and the policy space can be exhaustively explored.

In contrast, Q-Learning, SARSA, and Monte Carlo fail to achieve any meaningful reward. This may be attributed to the limited number of training episodes, insufficient exploration, or overly cautious learning strategies. Despite the simplicity of the environment, these algorithms require longer training or better-tuned parameters.

6.2 Custom Environment (100x100)

Despite the increased complexity and state space size, Value Iteration and Policy Iteration still manage perfect scores. This is possible due to the environment’s deterministic nature and the full availability of the transition model. However, such scalability comes with significant computational cost.

Model-free approaches such as Q-Learning, SARSA, and Monte Carlo again perform poorly. The sheer size of the state-action space makes learning from sparse rewards extremely difficult within the fixed number of episodes. These results highlight their dependence on extensive exploration to uncover optimal behavior.

6.3 Expanded Environment (100x100 + Teleport)

In this scenario, the presence of a teleportation action introduces additional complexity. Although the Value and Policy Iteration algorithms still achieve minimal rewards, their performance drops drastically due to the unpredictable effects of teleportation.

Model-free methods remain ineffective, yielding zero average reward. The stochasticity introduced by teleportation likely disrupts the convergence of Q-tables and the consistency required for Monte Carlo and SARSA learning updates.

7 Conclusion

This comparative study demonstrates that the optimal choice of RL algorithm is highly dependent on the environment scale and properties:

- **For small, deterministic settings:** Value and Policy Iteration provide optimal performance with minimal training time.
- **For large or stochastic environments:** Q-Learning, SARSA, and Monte Carlo require extensive tuning and longer exploration to show benefits.
- **For theoretical insights and pedagogy:** Monte Carlo methods serve as conceptually transparent models but are limited in practical effectiveness.

Future work could focus on adaptive exploration techniques, neural approximators, and more scalable variants such as Deep Q-Networks or Actor-Critic methods.